

MER Capture

Technical documentation and responsible-AI note · github.com/Kelzbot/mer-capture

IRCE AI Hackathon 2026 — Track 2: AI-Assisted Clinical Documentation

Converts free-text clinical consultation notes into structured, coded documentation with verifiable provenance and automated documentation-quality checks.

Live app: <https://kelzbot.github.io/mer-capture/> · **Pitch deck:** <https://kelzbot.github.io/mer-capture/pitch/>

PDFs for submission: [deck](#) · [documentation](#) — both regenerated from `README.md` and `pitch/index.html`, which remain the source of truth.

The problem

In Nigerian secondary and tertiary facilities the same clinical encounter is documented by hand three times: the clinician writes a free-text note, someone re-keys it into the EMR, and someone else codes the diagnosis for reporting. Each hop is manual and unverifiable.

The result is incomplete SOAP notes, diagnoses that never get coded, and treatment plans separated from the diagnosis that justifies them. That degrades continuity of care and corrupts facility reporting — which surfaces later as data quality findings against the facility.

Problem statement: Clinicians need a better way to convert free-text consultation notes into complete, coded, structured documentation, because manual transcription and coding is repeated, inconsistent and unverifiable, resulting in incomplete records and unreliable facility reporting.

The solution

INPUT	DE-IDENTIFY	AI	OUTPUT	HUMAN	ACTION
free-text narrative, pasted or one column of a CSV / XLSX	→ on this device before any network call: IDs, phone numbers and labelled or titled names → tokens	→ extraction to a fixed JSON schema, per-field confidence and a verbatim source span	→ structured SOAP note, ICD-10, medications, plan – then rule-based quality flags	→ clinician edits any field, highlights the source sentence, signs by name	→ confirmed record downloads as CSV (EMR / FHIR mapping is proposed)

Two things this diagram gets right that the earlier draft did not: de-identification is its own stage between input and the model, and the human gate lives in the **single-note view only** — batch mode is an analyst path for dataset-wide quality and evaluation, and has no per-record sign-off yet.

Why AI is justified

The input is unstructured free text containing clinical abbreviations, dictation artefacts and inconsistent phrasing. Keyword rules and regex section-splitting cannot reliably parse it. Extraction from natural clinical language is the one job that genuinely requires a language model.

The architectural line that matters

The LLM only reads and extracts. Every quality flag and every derived metric is computed by deterministic rules over the extracted fields.

No reportable output is ever generated by the model. This makes every flag auditable and means a hallucinated indicator is structurally impossible — the model never computes one.

What is WORKING / SIMULATED / PROPOSED

COMPONENT	STATUS	DEMONSTRATES	WHAT REMAINS
LLM extraction (narrative → fixed JSON schema)	WORKING	Per-field confidence and a verbatim source span, located back into the note by character offset	A scored run on the organiser dataset; broader schema coverage
Client-side de-identification	PARTIAL	Nigerian phone numbers, labelled hospital/file/patient IDs, slash-delimited codes, and names that are labelled (Name:) or titled (Mrs , Dr , Alhaji , Baby ...) become tokens before any network call	A bare name in running prose is not caught — needs a name lexicon or model. Formal privacy audit
Documentation quality rules (deterministic)	WORKING	Six rules, listed in full below	Clinician-validated rule set
Batch processing across a full dataset	WORKING	Three concurrent extractions, cancellable mid-run, one failed row does not stop the rest	Throughput and cost at facility volume
Evaluation harness vs reference fields	WORKING	Factual-match scoring on a seeded held-out split; metric functions verified against hand-built cases during the build	A run on the organiser dataset; the non-AI baseline; clinician adjudication
Named clinician sign-off gate	PARTIAL	Single-note view: export disabled until a clinician types their name and ticks that they checked every field	No per-record gate in batch mode
Transmission inspector	WORKING	The exact provider, model, endpoint, full prompt and de-identified payload that left the browser — plus the identifier map that stayed in the tab	Same view for batch runs
PEPFAR dataset mode (the engine this was adapted from)	WORKING	Column auto-mapping, MER indicators each carrying their reason , DQ rules with per-facility scoring	Outside this track's scope
EMR / FHIR export	PROPOSED	CSV download exists today	FHIR resource mapping — not started
OCR / photo capture of handwritten notes	PROPOSED	Nothing in the code accepts an image	A new input stage ahead of de-identification — not built

Disclosure

Required by the participant handbook, stated plainly:

- Pre-existing:** the extraction pipeline, client-side de-identification, and the deterministic rules/indicator engine were originally built for an HIV programme reporting use case and adapted to this track's dataset.
- Built during the event:** the clinical schema profile, the documentation-quality rule set, the evaluation harness, and the reframing of the application to this dataset.
- Third-party model: no model was trained or fine-tuned.** The only AI component is a hosted foundation model called over HTTPS. Two providers are wired up in `extract.ts` and picked at runtime: Google `gemini-2.5-flash` (default, called directly over REST) and Anthropic `claude-sonnet-4-6` (via `@anthropic-ai/sdk` with `dangerouslyAllowBrowser`). Neither is our model and we make no claim to either.
- Libraries**, exactly as in `package.json`. Runtime: `react` 19, `react-dom`, `papaparse`, `xlsx` (SheetJS), `lucide-react`, `@anthropic-ai/sdk`. Build and tooling: `typescript`, `vite`, `@vitejs/plugin-react`, `tailwindcss` 4 with `@tailwindcss/postcss`, `postcss`, `autoprefixer`, `oxlint`, `tsx`. All open source. No charting library — the bars are CSS.

Data

Dataset used: the organiser-provided Track 2 synthetic clinical encounter dataset. Columns include `clinical_narrative` (input) alongside reference structured fields (`chief_complaint` , `hpi` , `pmh` , `exam` , `differential` , `final_diagnosis` , `icd10` , `investigations` , `medications` , `treatment_plan` , `soap_note`).

Ground truth: the reference structured fields paired with each narrative. Extraction runs on the narrative only; the reference fields are used solely for scoring.

No real patient data was used at any point. The dataset is synthetic. De-identification runs client-side regardless, before any network call, as a defence-in-depth measure and because the deployment target is real clinical settings under the Nigeria Data Protection Act 2023.

Data handling: everything runs in the browser tab. There is no server of ours: nothing is uploaded to one, nothing is written to `localStorage` or `sessionStorage` , nothing persists after the tab closes. The API key is entered at runtime and held in React state only — but it is, necessarily, sent to whichever model provider you select, along with the de-identified note. In batch mode only the chosen note column is sent; the reference columns are never transmitted.

What de-identification actually does

Implemented in `deidentify.ts` , run on the text before the prompt is built. It replaces, in this order:

REDACTED	HOW IT IS RECOGNISED	TOKEN
Phone numbers	+234 / 0 prefixed candidates, kept only if the digits match a Nigerian pattern (234 + 10 digits, or 0[789][01] + 8)	[PHONE_n]
Labelled identifiers	A <code>hosp no</code> / <code>hospital number</code> / <code>file no</code> / <code>folder no</code> / <code>patient id</code> / <code>pid</code> label followed by its value	[ID_n]
Slash-delimited codes	Any token of three or more slash-separated segments, unless it is purely digits and slashes with fewer than three slashes	[ID_n]
Labelled names	A <code>name:</code> / <code>surname:</code> / <code>first name:</code> style label followed by its value	[NAME_n]
Titled names	A title (<code>Mr</code> , <code>Mrs</code> , <code>Miss</code> , <code>Ms</code> , <code>Mallam</code> , <code>Alhaji</code> , <code>Hajia</code> , <code>Chief</code> , <code>Dr</code> , <code>Prof</code> , <code>Baby</code>) followed by up to three capitalised words	[NAME_n]

The same original always maps to the same token, so co-reference inside a note survives.

What it does not catch, stated plainly: a bare personal name in running prose with no label and no title. Redaction is regex-based, not a named-entity model. Treat it as defence in depth, not as a guarantee of anonymity.

What happens to tokens in returned values: the map from token to original never leaves the tab. When the model returns a value or source span containing a token, `rehydrate()` substitutes the original back client-side, so the record you review and export reads correctly while the provider only ever saw the token. Values that survive as tokens because the model invented one are simply left as-is and are visible as such. The full de-identified payload, the prompt, and the token→original map are all displayed in the transmission inspector.

One known rough edge, unfixed and worth naming: the slash-code rule also tokenises ARV regimen strings such as `TDF/3TC/DTG` , because they are three slash-separated segments. The value round-trips correctly through `rehydrate()` , but the model sees a token instead of the regimen and cannot reason about it. This affects the PEPFAR profile, not the clinical one.

Documentation quality rules

Every rule below is a pure function in `docQuality.ts` operating on the extracted fields. None of them involves the model. This is the complete set — nothing is listed here that is not implemented.

RULE	SEVERITY	FIRES WHEN
<code>diagnosis_without_icd10</code>	critical	A final diagnosis is present but no ICD-10 code
<code>plan_without_diagnosis</code>	critical	A treatment plan is present with no diagnosis to justify it

RULE	SEVERITY	FIRES WHEN
<code>complaint_diagnosis_mismatch</code>	warning	The chief complaint and the final diagnosis share no keyword after stemming and stopword removal — a prompt to review, not a verdict
<code>medications_not_in_plan</code>	warning	Medications are listed but none of their names appears in the treatment plan
<code>incomplete_soap</code>	warning	Any core SOAP section is empty — one flag per missing section, across chief complaint, HPI, exam, differential, final diagnosis, treatment plan
<code>low_confidence_diagnosis</code>	warning	The final diagnosis or ICD-10 field was extracted with confidence below 0.5

Alongside the flags, `computeDocQuality` reports: records extracted, the share of records with every core section present, section fill rate per field, the diagnosis coding rate (diagnoses carrying an ICD-10 code), and mean extraction confidence per field and overall. Records are grouped for scoring by an explicit group or, failing that, by the prefix of the record ID — for the PEPFAR dataset mode, `dq.ts` scores by the real facility column instead.

Validation

Claim: MER Capture extracts structured clinical fields and ICD-10 codes from free-text consultation notes at a factual-match rate high enough to make clinician review faster than manual documentation, without inventing facts absent from the source.

Method: `evaluate.ts` pairs each extracted record with its reference row positionally, detects which reference columns are present, and scores only the fields that have one. The held-out set is drawn by a seeded Fisher–Yates shuffle (mulberry32 PRNG) — default 20%, seed 42, both adjustable and both displayed on screen with the resulting test-set size, so any reported number is reproducible. The extraction prompt lives in `extract.ts` and contains no dataset-specific examples.

Metrics, exactly as implemented:

METRIC	DEFINITION IN CODE	DENOMINATOR
Exact match	Equality after lowercasing, collapsing whitespace, stripping surrounding quotes and trailing punctuation. Reported macro-averaged across fields, and micro across all pairs	Pairs where the reference field is non-empty
Token F1	Multiset bag-of-words F1 with a stopword list applied, averaged per field then across fields	Same as above; an omitted field scores 0
ICD-10 exact	First code matching <code>[A-TV-Z][0-9][0-9A-Z](...)</code> in each side, compared in full	Records whose reference code parses
ICD-10 category	Same codes compared on the first three characters — <code>J45.9</code> counts as a category hit against <code>J45.0</code>	Same
Hallucination rate	Field populated where the reference field is empty	Field instances where the reference is empty
Omission rate	Field empty where the reference field is non-empty	Field instances where the reference is non-empty
Negation errors	Clause-split polarity check. Cues: <code>no</code> , <code>not</code> , <code>denies</code> , <code>denied</code> , <code>denying</code> , <code>without</code> , <code>absent</code> , <code>negative</code> , <code>nil</code> , <code>none</code> , each negating the next five content words within its clause. An error is a term the reference negates that the extraction asserts, or the reverse — counted only for terms present on both sides	Count, not a rate; examples listed with the record and term

`"NA"`, `"nil"`, `"none"`, `"-"`, `"unknown"`, `"not documented"` and similar are treated as empty on both sides, so a documented non-answer is not scored as content.

The harness also returns a per-record breakdown ranked worst-first, and the UI renders extracted against reference side by side for the worst cases so failures are inspectable rather than just counted. Everything exports as one CSV.

Results: not yet run against the organiser dataset. The harness is implemented, and its metric functions were checked during the build against hand-built extraction/reference pairs (exact match, token F1, ICD-10 exact and category, hallucination, omission, negation inversion, split reproducibility) — that check was a throwaway script and is

not committed. Producing the numbers below requires an API key and a batch run over the dataset. They are deliberately left blank rather than estimated.

METRIC	RESULT
Held-out set size	<i>fill from eval run</i>
Overall exact match	<i>fill</i>
Overall token F1	<i>fill</i>
ICD-10 exact	<i>fill</i>
ICD-10 category	<i>fill</i>
Hallucination rate	<i>fill</i>
Negation errors	<i>fill</i>

The same values fill the deck: `pitch/index.html` reads them from the query string, e.g. `pitch/index.html?TEST_SET_SIZE=40&EXACT_MATCH_PCT=86.4%25&FUZZY_F1=0.79`. Anything not supplied stays visibly marked as unfilled.

Success threshold: ≥85% field-level factual match and <2% hallucination rate would justify a supervised clinical pilot.

Failure criteria: hallucination rate above 5%, or any negation inversion on a safety-relevant field (allergy, medication, symptom denial), stops deployment pending redesign.

Baseline: the simplest non-AI alternative is regex/keyword section-splitting on the narrative. **Not measured.** We have not run that baseline on the same held-out set, so we make no claim that extraction beats it. Building and scoring that baseline is the next task, and until it is done the metrics above establish absolute performance only, not a margin over the cheap alternative.

Fairness: performance should be checked separately across age, sex and presenting condition. Whether the dataset permits this depends on those columns being populated — the harness scores any field with a reference column, so subgroup slicing is a matter of splitting the input rather than new code. Where a subgroup column is absent or sparse, the result is reported as unavailable rather than assumed equal.

Responsible AI and safety

OUR AI MUST NEVER generate a diagnosis, a code, or a clinical fact that is not present in the source note, or export any record without named clinician confirmation.

That rule is enforced in the single-note view today. Batch mode exports without a per-record gate — closing that is the first item on the list, and we would not run a clinical pilot until it is closed.

Safeguards, each one pointing at the code that implements it:

- **Confidence on every extracted field** — coerced to 0–1 in `extract.ts`, shown against every field in the UI, and used by the `low_confidence_diagnosis` rule.
- **Source span on every extraction** — the model must return a verbatim substring as evidence; `locateSpan()` finds its character offsets in the original text. In the single-note view, hovering a field highlights that sentence in the note, so verification is a glance rather than a re-read. Batch mode stores the spans but does not yet render them.
- **Unable-to-determine rather than a guess** — the prompt instructs the model to omit a field the note does not state, and `parseExtraction` drops empty values and rejects field names outside the selected profile. This is an instruction plus a filter, not a hard guarantee; the evaluation harness exists to measure how well it holds.
- **Named clinician sign-off** — single-note view only, as above: export stays disabled until a name is typed and the review box ticked.
- **Deterministic derived output** — every quality flag, indicator and metric is a pure function in `docQuality.ts`, `dq.ts`, `indicators.ts` or `evaluate.ts`. No number rendered anywhere in the app is produced by the model.
- **On-device de-identification** before any network call, with the limits stated honestly in the Data section above — regex-based, catches labelled and titled names but not bare names in prose.
- **Transmission inspector** — the provider, model, endpoint, complete prompt and de-identified payload that left the browser are all displayed, next to the identifier map that did not.

- **No persistence** — no `localStorage`, no `sessionStorage`, no server of ours, nothing retained after the tab closes.

When the AI is wrong: the source span makes the error visible at the point of review, low confidence is flagged, the clinician overrides, and the confirmed record — not the raw extraction — is what leaves the system.

Biggest assumption: that reviewing a pre-filled structured note is meaningfully faster than typing one, and that clinicians will actually correct low-confidence fields rather than rubber-stamp them.

Biggest risk: automation bias — a confident, fluent, wrong extraction being confirmed without scrutiny. This is why source spans are mandatory rather than optional.

Running it

```
npm install
npm run dev
```

Open the local URL. The header carries three mode tabs — **Dataset**, **Single note**, **Batch notes** — a provider selector (Google Gemini or Anthropic Claude) and the API key field. The key is held in memory only; it is never written to disk, and none is committed to this repository.

Batch notes is the flow for this track: upload a CSV/XLSX, pick the column holding the free-text narrative and the column holding the record ID, run the extraction, read the documentation-quality findings, and — if the file also carries the reference structured columns — the evaluation panel appears underneath with the held-out split control and the CSV export.

Single note is the flow that demonstrates the human gate: paste one note, watch the redaction happen, inspect exactly what was transmitted, edit any field, hover a field to highlight its source sentence, then sign off by name to enable export.

Dataset is the original PEPFAR mode: upload a CSV/XLSX of programme data, confirm the auto-mapped columns, and read MER indicators and data-quality findings. It needs no API key at all.

The pitch deck is a standalone file — open `pitch/index.html` in any browser; arrow keys or click to advance.

```
npm run build          # typecheck + production build
npx tsx src/lib/smoke.ts # parser and indicator assertions
```

Next 30-day experiment

A time-and-accuracy study with 5 clinicians documenting 20 encounters each — half manually, half by reviewing MER Capture output. Measuring documentation time, field completeness, coding accuracy, and clinician-corrected error rate.

That answers the only question that matters for deployment: does reviewing a machine-drafted note actually beat writing one, and does it beat it without introducing errors the clinician doesn't catch.

MER CAPTURE · DOCUMENTATION · APP <https://kelzbot.github.io/mer-capture/> · DECK <https://kelzbot.github.io/mer-capture/pitch/> · SOURCE <https://github.com/Kelzbot/mer-capture>